JjTL and JjEL: A Set-Theoretic Approach to Model Transformation and Expression Languages

Jjodel Project: Language Design Document

April 15, 2026

Abstract

This document presents the design and rationale of the domain-specific languages developed within the Jjodel ecosystem: JjEL (Jjodel Expression Language), a set-theoretic expression language inspired by first-order logic; JjTL (Jjodel Transformation Language), a declarative Model-to-Model transformation language; JjModal, a minimal interaction sub-language; JjLet, a let-binding construct for interactive transformations; and JjScript (Jjodel Scripting Language), an imperative command language for direct metamodel manipulation. We describe the syntax, semantics, and, most importantly, the *design motivations* behind each language construct, explaining *why* certain decisions were made and what alternatives were considered. We then provide a detailed comparison with four established transformation languages: ATL, EOL/ETL, QVT Relations, and QVT Operational Mappings. Finally, we present a series of parallel transformation examples across ATL, ETL, and JjTL to enable direct syntactic and conceptual comparison.

Contents

1 Introduction

Model-Driven Engineering (MDE) relies on model transformations as a fundamental mechanism for bridging abstraction levels, synchronizing representations, and generating artifacts from high-level specifications. Over the past two decades, a rich ecosystem of transformation languages has emerged, from the OMG's standardized QVT family to community-driven tools like ATL and Epsilon. Despite this maturity, adoption of dedicated transformation languages in industrial practice remains limited, with many practitioners preferring general-purpose languages like Java or Kotlin.

We identify three root causes for this adoption gap:

(i) **Cognitive overhead**: Existing languages require learning OCL, understanding complex execution semantics (e.g., ATL's two-phase execution, QVT-R's bidirectional pattern matching), and dealing with Eclipse-specific tooling.

(ii) **Syntax-semantics mismatch**: Languages like QVT-R use declarative syntax but require procedural reasoning about enforcement behavior; ATL mixes declarative bindings with imperative `do` blocks under different execution models.

(iii) **Expression language limitations**: OCL, while mathematically rigorous, was designed for *constraint specification* (what must be true), not for *value computation* (which elements, which values), yet it serves as the expression sub-language for ATL, QVT-R, and QVT-O.

The Jjodel project addresses these issues with two complementary languages:

- **JjEL**, a set-theoretic expression language whose core construct, `forall`, has *set comprehension* semantics (it returns a set, not a boolean). This bridges the gap between formal logic and practical computation, providing a single primitive that replaces `filter`, `map`, and logical quantification.
- **JjTL**, a declarative M2M transformation language that uses JjEL as its expression sub-language, offering a simpler rule structure than ATL while maintaining automatic traceability and guard conditions.
- **JjScript**, an imperative scripting language for direct metamodel manipulation, providing a command-based interface for creating, modifying, and querying metamodel elements with expression delegation to JjEL.

1.1 Design Criteria

Both languages were designed under three guiding principles, applied in strict priority order:

1. **Maximum declarativity.** The user describes *what* they want, never *how* to compute it. No assignment, no loops, no mutation.
2. **Minimum cognitive load.** Few rules to remember, minimal syntax, no boilerplate. The most frequent MDE operations are available as native methods.
3. **Maximum intuitiveness.** Constructs should read almost like natural language. Textual operators (`and`, `or`, `not`, `implies`), first-order-logic-inspired quantifiers (`forall`, `exists`), and null-safe navigation (`?.`, `??`).

1.2 Document Structure

Section 2 presents JjEL in detail, including its set-theoretic semantics and design motivations. Section 3 describes JjTL and its transformation model. Section 4 introduces JjModal, the interaction sub-language for user-facing commands. Section 5 presents JjLet, a let-binding

construct that bridges asynchronous interaction with synchronous expression evaluation. Section 6 presents JjScript, an imperative scripting language for direct metamodel manipulation. Section 7 provides a detailed comparative analysis with ATL, EOL/ETL, QVT-R, and QVT-O. Section 8 presents parallel transformation examples across multiple languages. Section 9 discusses open points and future directions.

2 JjEL: Jjodel Expression Language

JjEL is a pure, side-effect-free expression language designed for navigating, querying, and computing values over model graphs. Every JjEL expression produces a value; there are no statements.

2.1 Positioning and Scope

JjEL occupies a specific niche in the MDE language landscape:

OCL	JjEL	EOL	JavaScript
Formal constraints	Set-theoretic expressions	Model scripting	General-purpose
"what must be true"	"which elements, which values"	"manipulate the model"	"computation"

JjEL is *not* a constraint language (it computes values, not assertions), not a programming language (no variables, loops, or I/O), and not a transformation language (it provides expressions *inside* rules, not rules themselves). JjEL provides the expression sub-language for JjTL, analogous to how OCL provides expressions for ATL and QVT.

The scope boundary is precise: JjTL handles Model-to-Model (M2M) transformations; Model-to-Text (M2T) generation is handled by a separate template engine.

2.2 Types and Values

JjEL supports five primitive types and three composite types:

Category	Type	Literals	Examples
Primitive	`EString`	Double-quoted	`"hello"`, `"Customer"`
	`EInt`	Integer	`0`, `42`, `-7`
	`EDouble`	Decimal	`3.14`, `.5`
	`EBoolean`	Keyword	`true`, `false`
	`null`	Keyword	`null`
Composite	Array	Bracket	`[]`, `[1, 2, 3]`
	Object	(no literal)	Model elements
	Function	Lambda	`a => a.name`

The type system includes aliases for interoperability: `EString`/`String`, `EInt`/`Integer`, `EDouble`/`Double`/`Number`, `EBoolean`/`Boolean`. The `is` operator checks type membership including supertypes (kind-of semantics).

2.3 The Core Construct: `forall`

The most important design decision in JjEL is the semantics of `forall`.

2.3.1 Design Motivation

In classical first-order logic, the universal quantifier $\forall$ is an *assertion*: $\forall x \in S : P(x)$ returns a boolean (true if P holds for every x). In set theory, the set-builder notation $\{x \in S \mid P(x)\}$ *selects* elements, returning a set.

We chose **set-theoretic semantics**: `forall` returns a *set*, not a boolean. The rationale:

1. **Computational utility**: In model transformations, we almost always need to *select* elements and *compute* values from them. A set-returning construct is directly useful; a boolean-returning one requires additional machinery.
2. **Unification**: With set-theoretic `forall`, the `filter` and `map` collection methods become redundant; `forall` subsumes both. This reduces the number of concepts a user must learn.
3. **Natural reading**: "for all attributes such that they are public: get their name" reads naturally and maps directly to the syntax.

2.3.2 Syntax and Forms

`forall` has four forms, distinguished by three separators with distinct roles:

```
forall <var> in <collection> [such that <condition>] [: <projection>] [do <action>]
```

Form	Returns	Equivalent to	Example
`forall x in S such that P`	Set	`S.filter(P)`	`forall a in attrs such that`
`forall x in S: expr`	Set	`S.map(expr)`	`forall a in attrs: a.name`
`forall x in S such that P: expr`	Set	`S.filter(P).map(expr)`	`forall a in attrs such that`
`forall x in S do action`	–	Imperative loop	`forall a in attrs do create`

2.3.3 Separator Design

The three separators are *not* interchangeable; each has a precise role:

- `such that` introduces a *boolean filter condition*. It reads as a natural qualifier: "all attributes *such that* they are public."
- `:` introduces a *value projection*. It reads as "give me": "all attributes: their name" (i.e., the names of all attributes).
- `do` introduces an *imperative action*. It is reserved for JjTL transformation actions and is not available in pure JjEL expressions. This separation ensures JjEL remains side-effect-free.

The decision to use `such that` (two words) rather than a symbol like `|` was deliberate: it prioritizes readability and matches the set-builder notation commonly taught in discrete mathematics courses ($\{x \in S \mid P(x)\}$ is read "x in S *such that* $P(x)$").

2.3.4 Why Not `filter` and `map`?

The `filter`/`map` pair requires the user to:

1. Know two distinct method names and their order (filter first, then map).
2. Write a lambda for each: `attrs.filter(a => a.isPublic).map(a => a.name)`.
3. Understand method chaining and intermediate collections.

With `forall`, the same expression becomes:

```
forall a in attrs such that a.isPublic: a.name
```

This is one concept instead of three, with a single binding variable (`a`) instead of two lambdas. The cognitive load reduction is significant, especially for domain experts who are not programmers.

2.4 Existential Check: `exists`

While `forall` returns a set, `exists` returns a **boolean**: does at least one element satisfy the condition?

```
exists a in attributes such that a.type == "String"
exists a in attributes | a.isPublic and not a.isDerived
```

`exists` is semantically derived from `forall`:

$$\texttt{exists x in S such that P} \equiv \texttt{(forall x in S such that P).isNotEmpty}$$

For `exists`, the accepted separators are `such that` and `|` (pipe, as an alias). The `:` separator is **not** accepted for `exists`—it is reserved exclusively for `forall` projections. This avoids ambiguity in nested expressions such as `forall c in classes | (exists a in c.attrs | a.isPublic) : c.name`.

2.5 Logical Implication: `implies`

```
isAbstract implies subClasses.isNotEmpty
a.isPublic implies a.type != null
```

`implies` is semantically equivalent to `not P or Q` but reads as natural language. It is right-associative, short-circuit evaluated, and has lower precedence than `and`/`or`.

The motivation is direct: model validation invariants frequently express implications ("if a class is abstract, then it must have subclasses"). Without `implies`, users must negate the antecedent manually (`not isAbstract or subClasses.isNotEmpty`), which is less readable and more error-prone.

2.6 Context Binding: `with...do`

```
with selectedNode do name.pascalCase()
with selectedNode do forall a in attributes such that a.isPublic: a.name
```

The `with` construct establishes a context object whose properties become directly accessible in the body. This is essential in the Console REPL, where there is no implicit source element (unlike JjTL rules where the matched element provides implicit context).

The choice of `do` as the body separator (rather than a block or colon) was deliberate: it reads naturally ("with this node, *do* [evaluate] this expression") and parallels the imperative `do` in `forall` by indicating "what to compute in this context."

2.7 Conditional: `if...then...else`

```
if isAbstract then "abstract" else "concrete"
if lowerBound == 0 then "NULL" else "NOT NULL"
```

This construct is retained from common expression languages. An alternative syntax (`when...then...otherwise`) was considered for its more declarative flavor, but `if/then/else` was kept for its universality and zero learning curve.

When `else` is omitted, the expression returns `null` for the false branch, maintaining the invariant that every expression produces a value.

2.8 Lambda Expressions

```
a => a.name
(a, b) => a + b
```

Lambdas use the `=>` arrow syntax. An earlier design used `:` as the lambda separator (`a: a.name`), but this conflicted with the projection role of `:` in `forall`. The switch to `=>` freed `:` for its set-theoretic role and made the grammar LL(1)-parseable without backtracking.

Lambdas are primarily used with collection methods not subsumed by `forall`: `sortBy`, `groupBy`, `distinctBy`, etc.

2.9 Null Safety

```
parent?.name                    -- null if parent is null
parent?.name ?? "no parent"     -- "no parent" if chain is null
element?.container?.package?.name ?? "default"
```

Two operators handle null propagation:

- `?.` (null-safe navigation): returns `null` if the receiver is null, without throwing.
- `??` (null coalesce): returns the right operand if the left is null.

Using `.` on a null value throws an error by design: this forces users to explicitly acknowledge potential nulls via `?.`, making null handling visible rather than silently propagating.

2.10 Operator Precedence

Level	Construct	Associativity
1 (lowest)	`if ... then ... else ...`	right
2	`??` (null coalesce)	left
3	`implies`	right
4	`or`	left
5	`and`	left
6	`==`, `!=`	left
7	`<`, `>`, `<=`, `>=`	left
8	`is`	–
9	`+`, `-`	left
10	`*`, `/`, `%`	left
11	`not`, `-` (unary)	right
12 (highest)	`.`, `?.`, `[index]`	left

2.11 Built-in Methods

JjEL provides over 130 built-in methods organized by type. The guiding principle is that the most frequent MDE operations should be native, requiring no import or boilerplate.

Category	Count	Highlights
String	36	`camelCase()`, `pascalCase()`, `snakeCase()`, `kebabCase()`
Number	35	`abs()`, `clamp()`, `between()`, trigonometry
Collection	31	`sortBy()`, `groupBy()`, `flatten()`, `distinct()`
Date	36+5*	`addDays()`, `diffMonths()`, `format()`

*36 instance methods plus 5 constructor functions (`now()`, `today()`, `date()`, `datetime()`, `parseDate()`).

Naming convention methods (`camelCase`, `pascalCase`, `snakeCase`, `kebabCase`) are native because they are essential in model transformations (e.g., converting class names to table names, attribute names to column names). No other transformation language provides these natively.

Note: while `filter()` and `map()` remain available for backward compatibility and for use with lambda-based patterns, the idiomatic JjEL approach is to use `forall`, which subsumes both operations in a single construct. The remaining collection methods are those that `forall` does not subsume: aggregation (`sum`, `avg`), ordering (`sortBy`), structure (`groupBy`, `flatten`), and boolean checks (`all`, `any`, `none`).

2.12 Evaluation Rules

2.12.1 Short-Circuit Evaluation

`and` and `or` are specified with **short-circuit** semantics: the right operand should not be evaluated if the result is determined by the left operand alone. This enables guard-like expressions:[1]

```
parent != null and parent?.name.isNotEmpty    -- null-safe via ?. operator
```

2.12.2 Truthiness

JjEL uses a broad truthiness model: `null`, `false`, `0`, `""` (empty string), and `[]` (empty array) are all falsy. This enables concise guards (`if attributes then ...` instead of `if attributes.isNotEmpty then ...`).

2.12.3 Error Handling

JjEL distinguishes between recoverable and non-recoverable errors:

- **Silent null**: undefined variables, property not found on object (with console warning and typo suggestions via Levenshtein distance), division by zero, type mismatches in arithmetic.
- **Thrown error**: `.property` on `null` (forces use of `?.`), unknown method call (with suggestions).

3 JjTL: Jjodel Transformation Language

JjTL is a declarative, rule-based Model-to-Model transformation language. It uses JjEL as its expression sub-language.

3.1 Design Philosophy

JjTL targets the "sweet spot" between ATL's complexity and manual Java code:

[1] The current implementation (v0.x) evaluates both operands eagerly due to the structure of the binary expression evaluator. Short-circuit evaluation is specified as a target for a future release. In the meantime, null safety in guard expressions is best achieved via the `?.` operator.

- **Simpler than ATL**: No two-phase execution, no `thisModule` reference, no distinction between matched/lazy/called rules. Every rule is a class mapping.
- **More structured than ETL**: Attribute mappings use a declarative binding syntax (`:=`) rather than imperative assignment. Guards are explicit (`where`) rather than statement blocks.
- **JjEL instead of OCL**: Expressions use JjEL's set-theoretic semantics, null-safe navigation, and native naming conventions, features that OCL does not provide.

3.2 Transformation Structure

A JjTL transformation declares its name, source metamodel, target metamodel, and a set of class mappings and helper functions:

```
transformation StateMachine2PetriNet

from StateMachineMM
to   PetriNetMM

# Class mappings follow
State -> Place {
    tokens := if isInitial then 1 else 0
}

Transition -> Transition {
    name := name
}

# Helper functions
helper formatLabel(s: String) -> String {
    s.toUpper()
}
```

3.3 Class Mappings

A class mapping specifies a correspondence between one or more source classes and a target class:

```
SourceClass -> TargetClass [multiplicity] where { guard } {
    attributeMappings...
}
```

3.3.1 Multi-Source Mappings

A class mapping may specify multiple source classes, optionally with aliases, separated by commas:

```
ClassA a, ClassB b -> MergedClass where { a.name == b.name } {
    name := a.name
    description := b.description
}
```

This enables join-like semantics where multiple source instances are combined into a single target instance. Without aliases, source properties are accessed by class membership; with aliases, the alias provides an explicit qualifier for disambiguation.

3.3.2 Multiplicity

Multiplicity controls how many target instances are created per source instance:

- No multiplicity: one-to-one (default).
- `[*]`: unbounded (used with collection sources).
- `[n]`: exactly n target instances per source.
- `[m..n]`: between m and n instances.

3.3.3 Guard Conditions

The `where` clause filters which source instances are transformed. The guard is a JjEL expression evaluated with the source instance as implicit context:

```
Class -> Table where { not isAbstract and attributes.isNotEmpty } {
    ...
}
```

The implicit context means property access is unqualified: `isAbstract` refers to `source.isAbstract`. This reduces verbosity compared to ATL (where the guard must use the bound variable: `c.isAbstract`) and QVT-O (where `self.isAbstract` is required).

3.4 Attribute Mappings

3.4.1 Direct Mapping

The simplest form copies a value from source to target using the `:=` operator:

```
tableName := name
```

3.4.2 Value Mapping (Enum Conversion)

Discrete value conversions use the `:` separator followed by comma-separated pairs:

```
tokens := isInitial : true=1, false=0
code := status : "active"=1, "inactive"=0, "deleted"=-1
```

Unmapped values pass through unchanged (fallback identity).

3.4.3 Expression Mapping

Arbitrary JjEL expressions compute the target value:

```
tableName := name.snakeCase()
columnDefs := forall a in attributes: a.name + " " + a.type.toUpper()
```

3.4.4 Arrow Syntax (Alternative)

As an alternative to the `:=` assignment operator, JjTL also supports the `->` mapping arrow for attribute bindings:

```
name -> tableName
name -> tableName : name.snakeCase()
isInitial -> tokens : true=1, false=0
```

The `->` syntax reads as "source maps to target" and emphasises the directional nature of the mapping. Both syntaxes are interchangeable; `:=` is preferred for its brevity and natural reading when the target attribute name is clear from context, while `->` is useful when explicitly naming both the source property and the target property.

3.4.5 Object Creation

Nested target objects are created inline:

```
-> inputArcs {
    -> Arc {
        source := place
        weight := 1
    }
}
```

3.5 Helper Functions

Helpers are named, reusable JjEL expressions:

```
helper formatLabel(name: String, prefix: String) -> String {
    prefix + "_" + name.snakeCase()
}
```

Helpers are registered in the evaluation context and callable from any expression.

3.6 Trace Model

JjTL automatically records a trace model during execution, linking each source element to its target element(s) with binding-level detail:

- **Rule-level trace**: which class mapping produced which target instance.
- **Binding-level trace**: for each attribute mapping, the source value, target value, expression used, and whether the mapping is *invertible*.

3.6.1 Invertibility Analysis

JjTL automatically determines whether each attribute mapping can be reversed:

- Direct mappings (`b := a`): always invertible.
- Value mappings (`true=1, false=0`): invertible if and only if the mapping is injective (no duplicate target values).
- Expression mappings: invertible only for simple property references; function calls and arithmetic are marked non-invertible.

3.7 Interactive Features

JjTL supports user interaction during transformation execution via four built-in statements:

Construct	Behavior	Returns
`alert(msg, type)`	Blocking modal	–
`notify(msg, duration)`	Non-blocking toast	–
`prompt(msg, default)`	Text input dialog	String
`input(msg, type, options)`	Typed input dialog	Varies

These enable semi-automated transformations where certain decisions require human input, a feature absent from ATL, ETL, QVT-R, and QVT-O.

3.8 Execution Model

The JjTL executor follows a simple pipeline:

1. **Initialize**: bind source model elements in the evaluation context; register helpers.
2. **Match**: for each class mapping, find all source instances of the matching type.
3. **Guard**: filter instances through the `where` condition.
4. **Create**: instantiate target elements according to multiplicity.
5. **Bind**: evaluate attribute mappings and assign values.
6. **Trace**: record the trace link with invertibility information.

Unlike ATL's two-phase execution (where all bindings are computed before any imperative blocks run), JjTL processes each rule atomically: match, create, bind. This simplifies reasoning about execution order.

4 JjModal: Jjodel Interaction Language

JjModal is a minimal command language for human-in-the-loop model transformations. It provides a small set of *interaction commands* that, when evaluated, suspend execution, present a dialog to the user, and resume with the supplied value. JjModal commands are embedded transparently in JjTL transformation rules and JjScript expressions, following the same delegation pattern used by JjEL.

4.1 Motivation and Design Position

Model transformations are commonly described as fully automated processes: given a source model, a rule engine deterministically produces a target model. In practice, however, many real-world transformation scenarios involve decisions that cannot be encoded in the metamodel:

- A class name is ambiguous and the modeller must choose among several candidate target names.
- A transformation parameter (e.g., a database prefix, a version string) is only known at run time.
- A destructive action (delete all derived elements) requires explicit user confirmation before proceeding.

Existing transformation languages—ATL, ETL, QVT-R, QVT-O—provide no mechanism for this class of semi-automated transformation. The canonical workaround is to pre-configure transformations via external property files or to stop execution and restart after manual editing, both of which break the transformation workflow.

JjModal addresses this gap with a principled design governed by three criteria, listed in priority order:

1. **Minimal surface.** The language exposes the smallest possible number of commands. Each command corresponds to a distinct, irreducible interaction archetype: *ask for a value* (`prompt`) and *ask for a decision* (`confirm`).
2. **Transparent embedding.** Commands are recognised by name at the call site, without syntactic delimiters. A user reading a JjTL rule encounters `prompt(...)` in the same position as any other JjEL expression; no visual noise is added.
3. **Blocking semantics.** From the author's perspective, commands are synchronous: execution halts, the user responds, execution resumes with the typed value. The async/await implementation detail is hidden from the transformation author.

4.2 Commands

JjModal v0.1 defines two commands. Additional commands may be added in future versions following the same structural pattern.

4.2.1 `prompt`

```
prompt(label: EString, type: TypeRef, defaultValue?: any) -> any
```

`prompt` opens a modal dialog containing a labelled input field typed according to `type`. If `defaultValue` is supplied, the field is pre-populated with that value. The dialog blocks interaction until the user submits a value or cancels. On submit, the entered value is validated against `type` and returned to the caller. On cancel, the expression evaluates to `null`.

Design rationale. The command was named `prompt` rather than `input` to align with the established semantics of `window.prompt()` in browser environments, where the term denotes "ask the user for a string value," and to avoid collision with the `input` token used in JjTL transformation headers (`from MM`). The optional third argument, `defaultValue`, reduces friction in the common case where a reasonable default exists—the user can confirm it with a single keystroke.

4.2.2 `confirm`

```
confirm(label: EString) -> EBoolean
```

`confirm` opens a modal dialog containing the message `label` and two buttons labelled *Yes* and *No*. It returns `true` if the user selects *Yes* and `false` otherwise. The dialog is always dismissible via *No*; JjModal does not provide a non-dismissible confirmation.

Design rationale. `confirm` is kept strictly boolean. An earlier design considered a three-way `choose(label, options)` command for arbitrary enumerated choices, but this was deferred: the `prompt` command with a `MetaclassRef` type (see section 4.3) already covers the selection-from-set use case via a dropdown rendered by the UI layer. `confirm` is reserved exclusively for the binary yes/no decision archetype.

4.3 Type System

The `type` argument of `prompt` is drawn from two sources:

Ecore primitive types. The standard Ecore scalar types are accepted directly as type references:

Type	Input widget	Validation
`EString`	Text field	Always valid (empty string allowed)
`EInt`	Number field	`parseInt`; error if NaN or non-integer
`EFloat`	Number field	`parseFloat`; error if NaN
`EBoolean`	Checkbox	No validation required
`EDate`	Date picker	Valid ISO date required

Metamodel-relative references. An `IDENTIFIER` that is not a primitive Ecore type is resolved against the metamodel currently in scope. If the identifier refers to an `EEnum`, a dropdown is rendered with the enum literals as options. If it refers to an `EClass`, a selector is rendered over all model elements of that type currently loaded in the workspace. This enables prompts such as:

```
-- Ask the user to pick an existing Package from the source model
prompt('Target package', Package)
-- Ask the user to pick a Visibility literal
prompt('Visibility', VisibilityKind)
```

Validation in both cases is structural: the value returned is always a valid member of the declared type. The user cannot dismiss the dialog with *OK* while the field contains an invalid value; instead the input widget is decorated with an inline error indicator. Toast notifications are not used for validation errors, as they are easily missed and do not prevent submission.

Note: Metamodel-relative type resolution (`EEnum` and `EClass` references) is specified but not yet implemented in the current version. The `prompt` command currently accepts only Ecore primitive types (`EString`, `EInt`, `EFloat`, `EBoolean`, `EDate`). Metamodel-relative types are planned for a future release.

4.4 Grammar

Listing 1: JjModal formal grammar in EBNF

```
JjModalExpr  = PromptExpr | ConfirmExpr

PromptExpr   = 'prompt' '(' StringLit ',' TypeRef (',' DefaultExpr)? ')'

ConfirmExpr  = 'confirm' '(' StringLit ')'

TypeRef      = EcoreType | MetaclassRef

EcoreType    = 'EString' | 'EInt' | 'EFloat' | 'EBoolean' | 'EDate'

MetaclassRef = IDENTIFIER   (* resolved in metamodel scope at runtime *)

DefaultExpr  = Literal | JjELExpr   (* JjELExpr parsed by JjEL if not Literal *)

Literal      = NUMBER | StringLit | 'true' | 'false' | 'null'

StringLit    = '"' .* '"' | "'" .* "'"
```

The grammar is deliberately LL(1)-parseable without lookahead beyond the command name: `prompt` and `confirm` are both distinguishable at the identifier token. The `DefaultExpr` rule attempts literal parsing first; if the argument is not a literal, it is delegated to the JjEL parser.

This delegation strategy is identical to the one used when JjEL expressions appear inside JjTL bindings, and preserves the composability of the language family without introducing circular parser dependencies.

4.5 Embedding and Transparent Recognition

JjModal commands are not wrapped in any syntactic delimiter. The parser of the host language (JjTL or JjScript) recognises the tokens `prompt` and `confirm` followed by `(` as JjModal call sites and delegates parsing and evaluation accordingly.

The choice of transparent embedding over a delimited form (e.g., `$modal{...}` or `?{...}`) was motivated by two considerations. First, JjModal expressions appear in positions that already accept JjEL value expressions (attribute bindings, guard conditions, variable initialisers); adding a delimiter would create visual discontinuity at precisely the point where the syntactic context is already well-established. Second, the command names `prompt` and `confirm` are semantically unambiguous in the MDE domain: no JjEL built-in method and no JjTL structural keyword shares these names, making collision-free recognition reliable.

The only naming constraint imposed by this design is that JjModal command names are *reserved identifiers* in JjTL and JjScript: a metamodel element named `prompt` or `confirm` would be shadowed by the JjModal command. In practice this is not a concern, as both names are uncommon in Ecore metamodels; should a collision arise, the source element can be accessed via an explicit qualified reference.

4.6 Usage Examples

4.6.1 Prompt in an attribute binding

The most common use case is supplying a run-time value for a target attribute whose source has no direct counterpart:

Listing 2: JjModal prompt in a JjTL binding

```
transformation EnrichModel

from UML_MM
to   UML_MM

-- Add a database prefix to every Table, chosen at run time
Class -> Table where { not isAbstract } {
    name := prompt('Table prefix', EString, "tbl_") + name.snakeCase()
    -- The default "tbl_" is pre-filled; the user may override it
}
```

Note that the `defaultValue` argument `"tbl_"` is a JjModal literal; it could equally be a JjEL expression evaluated against the source element:

```
name := prompt('Table prefix', EString, owner.package?.name ?? "tbl_")
        + name.snakeCase()
```

Here `owner.package?.name ?? "tbl_"` is a JjEL expression (null-safe navigation and null coalesce) passed as the default value; it is evaluated once, before the dialog is opened, and its result pre-populates the field.

4.6.2 Prompt with a metamodel-relative type

Listing 3: JjModal prompt with MetaclassRef type

```
transformation MergePackages

from UML_MM
to   UML_MM

-- The user selects the target Package from a dropdown of all Package instances
Class -> Class {
    name := name
    package := prompt('Move to package', Package, owner)
}
```

4.6.3 Confirm in a guard condition

Listing 4: JjModal confirm as a guard

```
transformation PruneModel

from UML_MM
to   UML_MM

-- Only proceed with derived-attribute removal after explicit confirmation
Attribute -> Attribute
    where { not isDerived or confirm('Remove derived attribute ' + name + '?') } {
    name := name
    type := type
}
```

The `confirm` command is short-circuit evaluated here: if `isDerived` is `false`, the attribute is always transferred without interrupting the user. Only derived attributes trigger the dialog. This pattern uses JjEL's short-circuit `or` semantics to provide selective interactivity at no syntactic cost.

4.6.4 Confirm in JjScript

Listing 5: JjModal confirm in a JjScript block

```
-- Bulk delete: ask once before iterating
if confirm('Delete all derived attributes in the model?') then
    forall a in (forall c in classes: c.attributes) such that a.isDerived
    do a.delete()
end
```

4.7 Execution Semantics

4.7.1 Blocking from the author's perspective

The JjModal design contract is that commands are *synchronous* from the transformation author's perspective. The transformation text reads left-to-right, top-to-bottom; a `prompt` or `confirm` call produces a value that is immediately available to the enclosing expression. No explicit continuation, callback, or `await` keyword is required in the transformation source.

4.7.2 Async/await implementation

In the JavaScript/TypeScript runtime of Jjodel, true synchronous blocking of the UI thread is not available without web workers, and even then it would freeze the interface. JjModal is therefore implemented using **native async/await**: the JjTL and JjScript executors are async end-to-end, and each JjModal call site is compiled to an `await` on a `Promise` resolved by the modal dialog.

Listing 6: JjModal execution interface (TypeScript)

```
interface JjModalExecutor {
  prompt<T>(
    label:        string,
    type:         EType | MetaclassRef,
    defaultValue?: T
  ): Promise<T | null>

  confirm(
    label: string
  ): Promise<boolean>
}
```

The executor is injected into the JjTL/JjScript evaluation context at transformation startup. The transformation author is shielded from this interface entirely; they write `prompt(...)` and observe synchronous semantics.

4.7.3 Serialisation of interactions

Because the execution model is sequential (no parallel rule firing in JjTL v0.1), at most one JjModal dialog is open at any given time. The UI layer enforces this invariant: a second dialog cannot be opened while one is already active. If the executor were extended to support parallel rule evaluation in a future version, queuing or grouping of pending prompts would be required; this is an open design point noted in section 9.

4.8 UI Manifestation

All JjModal commands render through a single, centralised React component, `<JjModalDialog>`, mounted at the root of the application above all other layers. This architecture ensures:

- **Consistent visual treatment.** Every interaction command shares the same modal chrome (header, input area, action buttons), regardless of which rule or script triggered it.
- **No z-index conflicts.** Being mounted at the root, the dialog is always rendered above the transformation editor, property panels, and canvas.
- **Context display.** The dialog header can display the name of the transformation currently executing, providing the user with orientation during a multi-step semi-automated transformation.

The interaction channel between the executor and the React component is an *imperative handle*: the executor calls `modalChannel.openPrompt(config)` or `modalChannel.openConfirm(config)`, which sets component state and returns a `Promise`. The `Promise` is resolved by the component's submit/cancel handlers.

Command	Widget	Cancel behaviour
`prompt(..., EString)`	Text input	Returns `null`
`prompt(..., EInt)`	Number input	Returns `null`
`prompt(..., EFloat)`	Number input	Returns `null`
`prompt(..., EBoolean)`	Checkbox	Returns `null`
`prompt(..., EDate)`	Date picker	Returns `null`
`prompt(..., EEnum)`	Dropdown	Returns `null`
`prompt(..., EClass)`	Element selector	Returns `null`
`confirm(...)`	Yes / No buttons	Returns `false`

Cancel is always available and always produces a well-typed result (`null` for `prompt`, `false` for `confirm`), so the enclosing transformation expression is never left with an unresolved value. Transformation authors who need to distinguish "user provided null" from "user cancelled" can do so by checking the return value of `prompt` with JjEL's null-safe operators (`??`).

4.9 Relationship to the JjTL Interactive Feature Table

section 3 (section 3, §3.6) presents a preliminary table of interactive constructs (`alert`, `notify`, `prompt`, `input`) as JjTL built-in statements. JjModal formalises and narrows this set:

- `alert` and `notify` are *output* commands (they present information, not request it). They do not belong to JjModal, whose scope is strictly *input acquisition*. `alert` and `notify` remain JjTL-level built-ins and are not part of the JjModal grammar.
- `input` is subsumed by the typed `prompt` command: `prompt(label, type)` replaces both `input(label, type, options)` and the untyped `prompt(label, default)` from the preliminary table.
- `confirm` is a new command not present in the preliminary table; it formalises the binary decision pattern that previously had no dedicated construct.

Preliminary (§3.6)	Category	JjModal status	Note
`alert(msg, type)`	Output	Out of scope	Remains JjTL built-in
`notify(msg, duration)`	Output	Out of scope	Remains JjTL built-in
`prompt(msg, default)`	Input	→ `prompt`(label, type, default?)	Typed, extended
`input(msg, type, opts)`	Input	→ `prompt`(label, type, default?)	Merged
`confirm(label)`	Input	`confirm`(label)	New

4.10 Comparison with Related Work

No published M2M transformation language provides a dedicated interaction sub-language. The closest analogues are found in adjacent domains:

- **Wizard-based transformation tools** (e.g., Eclipse model wizards) externalise all user interaction into a preprocessing phase: the wizard collects parameters, then a conventional automated transformation runs. This clean separation avoids the complexity of mid-execution interaction but cannot express *per-element* decisions (e.g., confirm the removal of each derived attribute individually), only *global* parameters.
- **QVT-O** `assert` / `log` provide diagnostic output but no input facility. The modeller has no means to influence execution after the transformation starts.
- **ATL refining mode** allows transformations to be interrupted and restarted, but this is a debug mechanism, not an interaction design pattern.

JjModal's approach—embedding blocking interaction commands directly in the transformation rule language—is therefore a novel contribution to the transformation language design space, complementing the invertibility tracking and set-theoretic expression semantics that distinguish JjTL/JjEL from existing languages.

5 JjLet: Let-Binding for Interactive Transformations

JjLet extends the Jjodel language family with a *let-binding* construct that introduces lexically scoped, named variables into JjTL transformation rules and JjScript commands. While its design is general—the bound value can be any expression—its primary motivation is the combination of JjModal commands with the rest of the expression language: a `prompt` or `confirm` call is evaluated *once*, its result bound to a variable, and that variable is then available throughout the scoped body without re-prompting the user.

5.1 Motivation

Without JjLet, reusing the result of a JjModal command requires calling `prompt` multiple times, which opens the dialog once per occurrence:

```
-- Without let: dialog opened twice (undesirable)
Class -> Table where { not isAbstract } {
    name  := prompt('Column prefix', EString) + name.snakeCase()
    label := prompt('Column prefix', EString) + name.toUpper()
}
```

With JjLet, the dialog is opened once and the result is reused:

```
-- With let: dialog opened once, $p reused in two bindings
Class -> Table where { not isAbstract } {
    let $p = prompt('Column prefix', EString, 'col_') in {
        name  := $p + name.snakeCase()
        label := $p + name.toUpper()
    }
}
```

Beyond JjModal, JjLet is useful for naming any intermediate computed value, reducing repetition and improving readability:

```
let $derived = forall a in attributes such that a.isDerived in
    $derived.size > 0 implies $derived: a.name
```

5.2 Design Position and Architecture

JjLet occupies a precise architectural layer. The JjEL evaluator is *synchronous* by design, and JjModal commands are *asynchronous* (they await user input). Rather than making JjEL async—which would cascade through every caller in the system—JjLet is placed *above* JjEL: it lives in the JjTL executor and the JjScript executor, both of which are already async.

The execution model is:

1. The async executor encounters a `let` construct.
2. It `awaits` the evaluation of each binding expression (which may invoke JjModal via `UIBridge`).

3. It injects the resolved values into the JjEL evaluation context as named entries.
4. It delegates the body to JjEL, which evaluates it synchronously, looking up `$name` as a plain context variable.

From the transformation author's perspective, the execution is synchronous: `$p` is available immediately after the dialog closes, and the body evaluates as if `$p` had always been a constant.

Listing 7: JjLet execution flow

```
JjTL executor (async)
  let $p = prompt('Prefix', EString)
    await UIBridge.showPrompt(...)  -> "col_"
      ctx.child({ "$p": "col_" })
        JjEL evaluates body
          "$p" -> context lookup -> "col_"  [sync]
```

This design avoids any modification to the JjEL evaluator (`evaluator.ts`) and preserves full backward compatibility.

5.3 Variable Sigil

Variables introduced by `let` are always prefixed with `$`:

```
let $name = prompt('Name', EString) in ...   -- valid
let name  = prompt('Name', EString) in ...   -- parse error: missing $
```

The sigil serves two purposes. First, it provides a clear visual distinction between user-defined variables (`$name`) and metamodel properties (`name`), preventing silent shadowing of source element attributes. Second, it disambiguates variable references from identifiers in the JjEL expression grammar, which are always unqualified metamodel property names.

The `$` sigil requires a surgical extension to the JjEL lexer: the token sequence `$` *letter...* is recognised as a `DOLLAR_IDENT` token, which the JjEL parser treats as an identifier expression. This does not conflict with the existing string interpolation syntax `${expr}`, since `${` (dollar followed by brace) is recognised first.

5.4 Syntax

5.4.1 Single binding

```
let $identifier = expression in body
```

5.4.2 Multiple bindings

Multiple bindings are introduced with a comma-separated list and desugar to nested single bindings evaluated left-to-right. Later bindings may reference earlier ones:

```
let $x = e1, $y = e2 in body
-- desugars to:
let $x = e1 in let $y = e2 in body
```

```
-- Later binding references earlier one
let $base = prompt('Base name', EString),
    $full = $base + "_tbl" in
    forall c in classes such that c.name == $full
```

5.4.3 Block body in JjTL

In a JjTL mapping body, the `in` clause may be a brace-delimited block of attribute mappings:

```
let $p = prompt('Prefix', EString, 'col_') in {
    name  := $p + name.snakeCase()
    label := $p + name.toUpper()
    tag   := $p + "meta"
}
```

5.4.4 Formal grammar

Listing 8: JjLet grammar in EBNF

```
LetExpr      = 'let' BindingList 'in' LetBody

BindingList  = Binding (',' Binding)*

Binding      = '$' IDENTIFIER '=' expression

LetBody      = expression              -- in JjEL expression contexts
             | '{' MappingBody '}'     -- in JjTL binding block context
             | JjScriptCommand         -- in JjScript command context

expression   = LetExpr | ...           -- LetExpr is lowest precedence
```

`let` has the lowest precedence of all JjEL constructs (below `if/then/else` and `??`), so its body extends as far right as possible.

5.5 Semantics

5.5.1 Evaluation order

Bindings are evaluated **sequentially, left-to-right**. Each binding value is evaluated once; the result is immutable within the scope:

```
let $x = prompt('X', EInt), $y = $x * 2 in $x + $y
-- prompt shown once; $y computed immediately after
```

5.5.2 Lexical scope

The variable `$identifier` is visible **only within** `body`—it is not accessible outside the `let` expression. Re-binding via nested `let` is allowed and shadows the outer binding:

```
let $x = 1 in
let $x = $x + 1 in    -- inner $x = 2, outer $x = 1
    $x                -- returns 2
```

5.5.3 Cancellation handling

If the user cancels a `prompt`, the variable is bound to `null`. The body still executes; null-safe operators handle the absent value gracefully:

```
let $prefix = prompt('Prefix', EString) in
    forall c in classes: ($prefix ?? "") + c.name
```

If the user selects *No* on a `confirm`, the variable is bound to `false`:

```
let $ok = confirm('Proceed with deletion?') in
    if $ok then forall c in classes: c.delete() else null
```

5.6 Usage Examples

5.6.1 JjTL — shared prompt across bindings

Listing 9: JjLet with block body in JjTL

```
transformation EnrichSchema

from UML_MM
to   Relational_MM

Class -> Table where { not isAbstract } {
    let $prefix = prompt('Column prefix', EString, 'col_') in {
        name  := $prefix + name.snakeCase()
        label := $prefix + name.toUpper()
    }
}
```

5.6.2 JjTL — confirm before transformation

Listing 10: JjLet with confirm in a guard

```
Attribute -> Attribute
    where {
        let $ok = confirm('Map derived attribute ' + name + '?') in
            not isDerived or $ok
    } {
    name := name
    type := type
}
```

5.6.3 JjTL — multi-step interactive mapping

Listing 11: JjLet with multiple bindings

```
Class -> Table where { not isAbstract } {
    let $prefix = prompt('Table prefix', EString, 'tbl_'),
        $suffix = prompt('Table suffix', EString, '') in {
        name := $prefix + name.snakeCase() + $suffix
    }
}
```

5.6.4 JjScript — interactive bulk operation

Listing 12: JjLet in JjScript

```
let $target = prompt('Rename all instances to', EString) in
let $ok = confirm('Rename ' + (list Person).size + ' instances to '
                  + $target + '?') in
    if $ok then
        forall p in (list Person): rename p to $target
```

```
6     end
```

5.7 Implementation Status

JjLet was implemented in three phases, each building on the previous. All three phases are complete.

Phase 1 — `$identifier` token in JjEL lexer (✓). The only modification to the JjEL layer. The lexer recognises `$letter...` as a `DOLLAR_IDENT` token (before the existing bare-`$` error branch). The JjEL parser accepts `DOLLAR_IDENT` as a valid primary expression. The JjEL evaluator requires no changes: `$name` is looked up in the scope stack like any other identifier. Three files: `jjel/lexer/lexer.ts`, `jjel/types/tokens.ts`, `jjel/parser/parser.ts`.

Phase 2 — `let` in JjTL (✓). The JjTL parser recognises `let $x = e in { mappings }` as a mapping body item. The JjTL executor evaluates each binding asynchronously (enabling JjModal commands as binding values), injects results into a child JjEL context, and dispatches the body (expression or mapping block) within that context. Three files: `jjtl/types/ast.ts`, `jjtl/parser/parser.ts`, `jjtl/executor/executor.ts`.

Phase 3 — `let` in JjScript (✓). `let` is a command type in the JjScript command taxonomy. Its handler evaluates bindings asynchronously and executes the body command with an enriched context. Five files: `jjscript/types.ts`, `jjscript/parser/parser.ts`, `jjscript/executor/executor.ts`, `jjscript/executor/commands/let.ts`, `jjscript/index.ts`.

5.8 Design Tensions

`in` keyword collision. `in` is already used in JjEL quantifiers (`forall x in S`). In JjTL, the `parseJjELExpression()` function collects tokens up to a boundary set; `in` after a `let` binding must be recognised as the `let` separator rather than consumed as part of the binding expression. Since `let` is parsed at the JjTL layer (not inside JjEL's own grammar), the boundary rule can be extended without modifying the JjEL parser.

Context key convention. Variables must be stored in the JjEL context under a consistent key. Two conventions are possible: store `$name` as the key `"$name"` (sigil kept), or strip the sigil and store as `"name"`. Keeping the sigil is simpler (no transformation required) and avoids any risk of collision with metamodel property names—which are always sigil-free.

Block body boundary. The `{ mappings }` body is a JjTL-specific construct. The JjTL parser must distinguish `let $x = e in { ...}` (block body) from `let $x = { ...} in expr` (object literal as binding value, if supported). The disambiguation is straightforward: after parsing the `in` token, if the next token is `{` and the context is a mapping body, a block body is assumed.

6 JjScript: Jjodel Scripting Language

JjScript is an imperative, command-based scripting language for direct metamodel manipulation. While JjEL evaluates expressions without side effects and JjTL produces new target models declaratively, JjScript *mutates* the metamodel in place: it creates, renames, moves, and deletes metamodel elements as immediate operations on the live model graph.

6.1 Motivation and Design Position

The Jjodel language family assigns each language a distinct role along the *purity–effect spectrum*:

	JjEL	JjTL	JjScript
Paradigm	Functional	Declarative	Imperative
Side effects	None	None (new model)	Yes (in-place mutation)
Primary use	Querying, computing	M2M transformation	Metamodel editing
Operates on	Model instances	Source → target	Metamodel itself

JjScript fills a gap that neither JjEL nor JjTL can address: *ad hoc structural modification* of a metamodel during an interactive modelling session. Typical tasks include adding a class, renaming an attribute, setting a property to a new value, or bulk-updating elements matching a criterion. These operations are inherently imperative—they change state—and do not fit the declarative paradigm of JjTL or the pure expression semantics of JjEL.

The design is governed by three principles, in priority order:

1. **Command-oriented syntax.** Every JjScript statement begins with a verb (`create`, `rename`, `delete`, etc.) followed by its arguments. This mirrors the mental model of a user performing discrete editing actions and enables straightforward autocompletion.
2. **Expression delegation.** All value expressions—filters, computed defaults, collections—are delegated to JjEL. This avoids duplicating an expression evaluator and ensures that the same expression syntax works across all Jjodel languages.
3. **Metamodel scope.** JjScript operates on the *metamodel level* (M2): it creates and modifies classes, attributes, references, packages, and enumerations. It does not operate on model instances (M1), which are the domain of JjTL.

6.2 Language Architecture

JjScript follows a classic interpreter pipeline:

1. **Lexer** (`parser/lexer.ts`): tokenises the input into 16 token types, including `COMMAND`, `KEYWORD`, `IDENTIFIER`, `QUALIFIED_NAME`, `STRING`, `NUMBER`, `BOOLEAN`, `MULTIPLICITY`, `OPERATOR`, `ARROW`, and `COMMENT`. Keywords are case-sensitive and lowercase only; PascalCase tokens are classified as identifiers, allowing class names like `Attribute` to coexist with the keyword `attribute`.
2. **Parser** (`parser/parser.ts`): a recursive descent parser that produces a `CommandNode` AST. Each node carries the command type and a typed argument structure specific to the command.
3. **Executor** (`executor/executor.ts`): dispatches to a command-specific handler. Each handler validates arguments, resolves target elements in the metamodel, and performs mutations via Redux actions wrapped in `TRANSACTION()` calls. All handlers are asynchronous, enabling integration with JjModal for interactive commands.
4. **Result**: every command returns an `ExecutionResult` containing success status, a human-readable message, optional data payload, and error details with suggestions.

6.2.1 Element Resolution

A critical component is the **element resolver** (`executor/resolvers.ts`), which locates metamodel elements by qualified name. Three resolution strategies are applied in order:

1. **Metamodel-scoped** (preferred): searches only within the target metamodel specified by the execution context, preventing accidental cross-metamodel references.
2. **Project-wide**: searches all metamodels in the project, used when no target metamodel is specified.
3. **Name-only**: case-insensitive lookup by the last segment of the qualified name, with recursive package traversal.

Qualified names use the `::` separator for package paths (`com::model::Person`) and `.` for member access (`Person.name`).

6.2.2 Natural Language Normalisation

JjScript includes a **normaliser** (`normalizer/`) that converts natural-language or pseudo-syntactic input into formal JjScript commands via pattern-based rules. A companion **detector** (`normalizer/detector.ts`) classifies input as JjScript, JjEL, or free text. This normalisation layer serves as the bridge between the AI assistant (Jjodie) and the JjScript executor: when a user types a natural-language request in the chat, Jjodie generates JjScript commands that the normaliser validates and the executor runs.

6.3 Command Taxonomy

JjScript provides 19 commands organised into four functional categories.

6.3.1 Structural Commands

These commands create and modify the metamodel structure.

Command	Syntax	Description
`create`	`create <type> <name> [in <parent>] [options]`	Creates a new metamodel element. Supports 15 element types (class, abstract class, interface, attribute, reference, containment, composition, operation, parameter, package, enum, literal, annotation, model, metamodel). Element-specific options include `type`, `[multiplicity]`, `default`, `abstract`, `extends`, `opposite`, `derived`, `readonly`.
`delete`	`delete [<type>] <target> [cascade|force]`	Removes an element. `cascade` deletes children; `force` ignores dependencies.
`rename`	`rename [<type>] <target> [in <parent>] to <name>`	Renames an element with conflict checking.
`set`	`set <target>.<prop> [=|+=|-=] <value>`	Sets a property. Operators: `=` (replace), `+=` (append), `-=` (remove from collection).
`add`	`add <type> <name> to <target>`	Syntactic sugar for `create` with a parent context.
`remove`	`remove <target> from <parent>`	Removes an element from a collection. Special form: `remove extends from <class>` clears inheritance.
`move`	`move <target> to <destination>`	Relocates an element, validating move legality.
`copy`	`copy <target> to <dest> [as <name>] [deep]`	Duplicates an element. `deep` copies nested children.
`extends`	`<Child> extends <Parent>`	Sets inheritance with circular-inheritance checking.

6.3.2 Inspection Commands

These commands query the metamodel without modifying it.

Command	Syntax	Description
`list`	`list [<type>] [in <scope>]`	Lists elements with optional type and scope filtering.
`show`	`show <target> [brief|full|tree]`	Displays detailed element information at three verbosity levels.
`validate`	`validate [<target>|all]`	Checks model correctness: name conflicts, type consistency, multiplicity satisfaction, circular inheritance. Returns categorised errors, warnings, and informational messages.

6.3.3 Control Commands

Command	Syntax	Description
`undo` / `redo`	`undo [n]` / `redo [n]`	Stack-based transaction reversal (up to 20 actions).
`clear`	`clear [history|console|selection]`	Resets command history, console output, or current selection.
`help`	`help [<topic>]`	Context-sensitive help for commands, element types, and syntax.

6.3.4 Advanced Commands

Command	Syntax	Description
`eval`	`eval <jjel-expression>`	Evaluates a JjEL expression in the current metamodel context and displays the result. Also triggered implicitly when the input begins with a JjEL keyword (`exists`, `with`, `forall`).
`let`	`let $var = <expr> [, $v2 = <e2>] in <command>`	Scoped variable bindings (see section 5). Supports `prompt()` and `confirm()` as binding expressions via JjModal integration.
`forall`	`forall <var> in <coll> [such that <pred>] do <cmd>`	Iteration: evaluates a JjEL collection expression, optionally filters with a predicate, and executes the body command once per matching element.

6.4 Type System

JjScript commands operate on 15 element types, mirroring the Ecore metamodelling constructs:

Category	Element Types	Notes
Classifiers	`class`, `abstract class`, `interface`	Structural types
Features	`attribute`, `reference`, `containment`, `composition`	Typed with multiplicity
Behavioural	`operation`, `parameter`	Method signatures
Packaging	`package`	With `nsUri`, `nsPrefix`
Enumeration	`enum`, `literal`	Discrete value types
Annotation	`annotation`	Metadata
Structural	`model`, `metamodel`	Top-level containers

Type references in `create` and `set` commands accept Ecore primitive types (`EString`, `EInt`, `EFloat`, `EBoolean`, `EDate`), qualified class names, enum references, and collection types (`List<T>`, `Set<T>`, `OrderedSet<T>`, `Bag<T>`, `Sequence<T>`).

Multiplicities follow EMF notation: `[0..1]`, `[1..*]`, `[*]`, or shorthand `?` (optional), `+` (one-or-more), `*` (unbounded).

6.5 Expression Delegation to JjEL

JjScript does *not* contain its own expression evaluator. All value expressions are delegated to JjEL via the `jjelEval()` function, following the same pattern used by JjTL (see section 3).

Three commands perform JjEL delegation:

1. `eval`: builds a JjEL evaluation context containing all metamodel classes, attributes, and project metadata, then calls `jjelEval(expression, context)`. The result is formatted for display (arrays, objects, and primitives with type annotations).
2. `let`: evaluates each binding expression via `jjelEval()` (or via JjModal's `UIBridge` for `prompt()`/`confirm()` calls). Resolved values are injected into a child context as `$`-prefixed variables, then the body command executes with the enriched context.
3. `forall`: evaluates the collection expression via `jjelEval()`, which must return an array. The optional `such that` predicate is evaluated per element with the iteration variable bound in a child context. The body command executes once per passing element with variable substitution.

This delegation architecture ensures expression consistency: the same JjEL expression that works in a JjTL binding or in the Console REPL works identically inside a JjScript `eval`, `let`, or `forall` command.

6.6 Autocompletion

JjScript includes a context-sensitive autocompletion engine (`autocomplete/engine.ts`) that provides real-time suggestions as the user types. The engine combines three providers:

1. **Keyword provider**: suggests command names, keywords, and element types based on the current cursor position and partial input.
2. **Metamodel provider**: suggests class names, attribute names, and reference names from the metamodel currently in scope.
3. **Type provider**: suggests primitive types and user-defined classes when the cursor is in a type-reference position.

Suggestions are ranked by relevance (exact prefix matches and recent commands are boosted) and presented with type annotations (`command`, `keyword`, `element`, `type`, `property`, `value`) to disambiguate visually.

6.7 Grammar

The following EBNF grammar is derived from the recursive descent parser (`parser/parser.ts`). It covers the core command syntax; element-specific options (e.g., `abstract`, `extends`, `type`) are parsed contextually by the `create` handler.

Listing 13: JjScript formal grammar in EBNF

```
Program         = Command*

Command         = CreateCmd | DeleteCmd | RenameCmd | SetCmd
                | AddCmd | RemoveCmd | MoveCmd | CopyCmd
                | ExtendsCmd | ListCmd | ShowCmd | ValidateCmd
                | HelpCmd | UndoCmd | RedoCmd | ClearCmd
                | EvalCmd | LetCmd | ForallCmd

CreateCmd       = 'create' ElementType IDENTIFIER
```

```
                    ('in' QualifiedName)?
                    CreateOptions*

DeleteCmd       = 'delete' ElementType? QualifiedName
                    ('cascade' | 'force')?

RenameCmd       = 'rename' ElementType? QualifiedName
                    ('in' QualifiedName)? 'to' IDENTIFIER

SetCmd          = 'set' QualifiedName '.' IDENTIFIER
                    ('=' | '+=' | '-=') Value

AddCmd          = 'add' ElementType IDENTIFIER 'to' QualifiedName

RemoveCmd       = 'remove' QualifiedName 'from' QualifiedName
                | 'remove' 'extends' 'from' QualifiedName

MoveCmd         = 'move' QualifiedName 'to' QualifiedName

CopyCmd         = 'copy' QualifiedName 'to' QualifiedName
                    ('as' IDENTIFIER)? ('deep')?

ExtendsCmd      = IDENTIFIER 'extends' IDENTIFIER
                | 'extends' IDENTIFIER IDENTIFIER

ListCmd         = 'list' ElementType? ('in' QualifiedName)?

ShowCmd         = 'show' QualifiedName ('brief' | 'full' | 'tree')?

ValidateCmd     = 'validate' (QualifiedName | 'all')?

EvalCmd         = 'eval' JjELExpression
                | JjELExpression     (* bare JjEL, auto-detected *)

LetCmd          = 'let' BindingList 'in' Command

ForallCmd       = 'forall' IDENTIFIER 'in' JjELExpression
                    ('such' 'that' JjELExpression)?
                    'do' Command

BindingList     = Binding (',' Binding)*
Binding         = '$' IDENTIFIER '=' (JjELExpression | ModalExpr)
ModalExpr       = 'prompt' '(' STRING ',' TypeRef (',' Value)? ')'
                | 'confirm' '(' STRING ')'

ElementType     = 'class' | 'abstract' 'class' | 'interface'
                | 'attribute' | 'reference' | 'containment'
                | 'composition' | 'operation' | 'parameter'
                | 'package' | 'enum' | 'literal'
                | 'model' | 'metamodel' | 'annotation'

QualifiedName   = IDENTIFIER ('::' IDENTIFIER)* ('.' IDENTIFIER)?

TypeRef         = PrimitiveType | QualifiedName
                | CollectionType '<' TypeRef '>'
PrimitiveType   = 'EString' | 'EInt' | 'EFloat' | 'EBoolean'
                | 'EDate' | 'String' | 'Integer' | 'Boolean'
CollectionType = 'List' | 'Set' | 'OrderedSet' | 'Bag' | 'Sequence'
```

```
68
69 Multiplicity   = '[' INT ('..' (INT | '*'))? ']'
70                | '*' | '+' | '?'
71
72 Value          = STRING | NUMBER | 'true' | 'false' | 'null'
73                | QualifiedName | JjELExpression
```

The grammar is LL(1) with one key ambiguity: bare JjEL expressions (without the `eval` prefix) are detected by checking whether the first token is a JjEL keyword (`exists`, `with`, or contextually `forall`). If so, the entire input is delegated to the JjEL parser. The `forall` keyword is disambiguated by context: if followed by an identifier, `in`, a collection, and `do`, it is a JjScript iteration; otherwise it is a JjEL set comprehension.

6.8 Interaction with Jjodie

JjScript serves as the **execution target** for Jjodie, the Jjodel AI assistant. The integration follows a three-layer architecture:

1. **Chat layer**: the user types a natural-language request ("add a name attribute of type String to Person").
2. **AI layer**: Jjodie generates one or more JjScript commands (`create attribute name in Person type EString`).
3. **Execution layer**: the `JjScriptService` detects that the message is a JjScript command (via the normaliser's detector), parses it, executes it, and returns the result formatted for the chat UI.

A design tension arises from keyword–natural-language overlap: words like `create`, `delete`, and `move` are both JjScript commands and common English words. The normaliser resolves this by applying pattern matching: if the input matches a known JjScript command pattern (verb + element type + arguments), it is classified as JjScript; otherwise it is treated as free text and forwarded to the AI layer.

6.9 Usage Examples

6.9.1 Metamodel Construction

Listing 14: Building a metamodel from scratch in JjScript

```
# Create structural types
create class Person
create abstract class BaseEntity
create interface Nameable

# Set inheritance
Person extends BaseEntity
Person extends Nameable

# Add features
create attribute name in Person type EString [1]
create attribute age in Person type EInt default 0
create reference friends in Person type Person [*]
create attribute email in Person type EString [0..1]

# Create an enumeration
create enum VisibilityKind
```

```
create literal PUBLIC in VisibilityKind
create literal PRIVATE in VisibilityKind
create literal PROTECTED in VisibilityKind

# Create a package and move elements
create package com.example nsUri "http://example.com" nsPrefix "ex"
move Person to com.example
```

6.9.2 Querying and Validation

Listing 15: Inspection and validation in JjScript

```
# List all classes
list class

# Show detailed view of a class
show Person full

# Validate the entire metamodel
validate all

# Evaluate a JjEL expression
eval forall c in classes such that c.isAbstract: c.name

# Query with exists
eval exists a in Person.attributes such that a.type == "EString"
```

6.9.3 Bulk Operations with `forall` and `let`

Listing 16: Advanced JjScript with iteration and bindings

```
# Set all String attributes to readonly
forall a in attributes such that a.type == "EString"
    do set a.readonly = true

# Interactive rename with prompt
let $name = prompt('New class name', EString) in
    rename class Person to $name

# Conditional bulk delete with confirm
let $ok = confirm('Delete all derived attributes?') in
    forall a in attributes such that a.isDerived
        do delete a
```

6.10 Relationship to Other Jjodel Languages

JjScript completes the Jjodel language family by providing the imperative, side-effecting component that the other languages deliberately exclude. Table 1 summarises the relationships.

Table 1: Jjodel language family: positioning of JjScript.

Aspect	JjEL	JjTL	JjScript
Paradigm	Functional	Declarative	Imperative
Side effects	None	None (new model)	Yes (in-place)
Operates on	Model instances (M1)	Source → target (M1)	Metamodel (M2)
Uses JjEL	Is JjEL	Yes (sub-language)	Yes (sub-language)
Uses JjModal	No	Yes	Yes (via `let`)
Uses JjLet	No	Yes	Yes
AI integration	No	No	Yes (via Jjodie)
Autocompletion	No	Planned	Yes (3 providers)
Undo support	N/A	N/A	Yes (20-action stack)

A key design invariant is that JjScript *never duplicates* expression evaluation logic: every expression inside a `let`, `forall`, or `eval` command is handled by JjEL, and every interaction command (`prompt`, `confirm`) is handled by JjModal. JjScript's own contribution is strictly the command dispatch, element resolution, and metamodel mutation layer.

6.11 Implementation Status

JjScript is fully implemented with approximately 4,500 lines of TypeScript across 50 files. The implementation includes:

- 19 command handlers, each in a separate module under `executor/commands/`.
- A three-strategy element resolver with metamodel-scoped, project-wide, and name-only fallback.
- A context-sensitive autocompletion engine with three providers (keyword, metamodel, type).
- A pattern-based normaliser for natural-language input.
- A chat integration service (`JjScriptService`) bridging JjScript with the Jjodie AI assistant.
- Error handling with Levenshtein-based fuzzy matching for typo suggestions.

7 Comparative Analysis

We now compare JjTL/JjEL with four established transformation languages, examining both syntactic design and underlying philosophy.

7.1 ATL (Atlas Transformation Language)

7.1.1 Overview

ATL is a hybrid declarative/imperative M2M transformation language developed at INRIA as part of the ATLAS group. It uses OCL as its expression sub-language and has been the most widely used academic transformation language since the mid-2000s.

7.1.2 Structural Comparison

Aspect	ATL	JjTL
Header	`module Name; create Out from In`	`transformation Name \n from X \n to Y`
Rule type	Matched / Lazy / Called / Entrypoint	Single type: class mapping
Binding	`targetAttr <- expr`	`targetAttr := expr`
Guard	OCL expression in `from` clause	`where` block with JjEL
Trace resolution	Implicit via matched rules	Implicit (like ATL) with per-binding invertibility
Imperative block	`do { ... }` (post-binding)	`do` in `forall` (per-element)
Expression language	OCL	JjEL

7.1.3 Key Design Differences

Binding direction. ATL uses `<-` ("target receives from expression"), while JjTL uses `:=` as primary syntax ("target is assigned from expression") and `->` as an alternative ("source maps to target"). The `:=` syntax is concise and familiar, while the `->` variant makes the directional nature of the mapping explicit.

Rule taxonomy. ATL distinguishes four rule types (matched, lazy, called, entrypoint), each with different execution semantics. JjTL has a single rule type with optional multiplicity and guard, reducing the conceptual surface.

Two-phase execution. ATL's matched rules execute in two phases: first, all target elements are created and bindings computed (declarative phase); then, `do` blocks run (imperative phase). This design enables inter-rule references during binding but makes execution order non-obvious. JjTL processes each mapping atomically: match, create, bind.

Expression language. ATL uses OCL, which lacks null-safe navigation, naming convention methods, and set comprehension via `forall`. OCL collection operations (`->select()`, `->collect()`) require lambda syntax (`e | expr`) and explicit collection type management (`Set`, `Sequence`, `Bag`, `OrderedSet`).

7.2 EOL / ETL (Epsilon Platform)

7.2.1 Overview

EOL is the imperative base language of the Epsilon platform; ETL extends it for M2M transformations. EOL takes a pragmatic approach, designed to be familiar to Java/JavaScript developers.

7.2.2 Structural Comparison

Aspect	ETL	JjTL
Rule syntax	`rule Name transform s:T to t:T { ... }`	`S -> T { ... }`
Binding	Imperative assignment: `t.x = s.y`	Declarative: `x := y`
Guard	`guard: expr` or `guard { block }`	`where { expr }`
Trace resolution	`::=` operator / `equivalent()`	Implicit with per-binding invertibility
Rule modifiers	`@abstract`, `@lazy`, `@primary`, `@greedy`	Multiplicity (`[*]`)
Inheritance	`extends parentRule`	Not supported
Control flow	`for`, `if/else`, `while`, `switch`	`forall`, `if/then/else`

7.2.3 Key Design Differences

Declarative vs. imperative bindings. ETL attribute mappings are imperative assignments (`t.name = s.name.toUpperCase()`). JjTL attribute mappings are declarative bindings (`tableName := name.toUpper()`). The declarative approach enables automatic invertibility analysis, which is impossible with imperative assignments.

Verbosity. ETL requires explicit variable declarations, `new` for object creation, and method call syntax for collection operations. JjTL infers targets from the rule header, creates objects implicitly, and uses `forall` instead of explicit loops or lambda chains.

Rule inheritance. ETL supports `extends` for rule inheritance (abstract rules with shared logic). JjTL does not have rule inheritance, favoring flat, self-contained rules. This is a deliberate simplicity trade-off.

7.3 QVT Relations (QVT-R)

7.3.1 Overview

QVT-R is the declarative relational sub-language of the OMG QVT standard. Its distinguishing features are bidirectional specification and pattern-based relation declarations.

7.3.2 Structural Comparison

Aspect	QVT-R	JjTL
Paradigm	Fully declarative, bidirectional	Declarative, bidirectional (with unidirectional extension)
Rule structure	`relation` with domain patterns	Class mapping with attribute bindings
Directionality	`checkonly` / `enforce` per domain	Bidirectional core; unidirectional extension
Precondition	`when { ... }` (relation calls)	`where { JjEL expr }`
Postcondition	`where { ... }` (relation calls)	Not explicit (sequential)
Expression language	OCL	JjEL
Variable binding	Implicit via pattern variables	Explicit via source property names

7.3.3 Key Design Differences

Bidirectionality. QVT-R's core promise is that the same transformation can be executed in either direction. In practice, this creates significant complexity: users must reason about enforcement behavior in both directions simultaneously, and non-bijective mappings (common in real transformations like class-to-table) introduce semantic ambiguities. JjTL's declarative core is also bidirectional, but takes a different approach: invertibility is tracked per-binding rather than per-transformation, giving fine-grained control over which parts of a mapping can be reversed. Additionally, JjTL provides a unidirectional extension for cases where source-to-target-only semantics are needed.

Pattern matching vs. explicit mapping. QVT-R uses pattern-based domain declarations where variables are bound across domains by name equality. This is elegant but hard to read: understanding which variables bind to what requires scanning the entire relation. JjTL uses explicit `->` mappings, making the data flow immediately visible.

Relation calls. QVT-R's `when`/`where` clauses invoke other relations, creating a web of interdependent correspondences. This compositional power is also a source of complexity: debugging requires tracing through chains of relation calls. JjTL guards are self-contained JjEL expressions.

7.4 QVT Operational (QVT-O)

7.4.1 Overview

QVT-O is the imperative sub-language of the QVT standard. It uses `mapping` operations with automatic trace management, OCL-based expressions, and a three-section body (`init`/population/`end`).

7.4.2 Structural Comparison

Aspect	QVT-O	JjTL
Entry point	Explicit `main()`	Implicit (all rules fire)
Mapping syntax	`mapping T::op() : TT`	`S -> T { ... }`
Implicit context	`self`	Unqualified (source properties)
Object creation	`object Type { ... }`	Nested `-> Type { ... }`
Guard	`when { ... }`	`where { ... }`
Trace resolution	Implicit via mapping invocation	Implicit with per-binding invertibility
Assignment	`:=`	`:=` and `->`
Expression language	OCL	JjEL

7.4.3 Key Design Differences

Entry point. QVT-O requires an explicit `main()` that orchestrates the transformation by navigating the source model and invoking mappings. JjTL fires all class mappings automatically for matching source instances (pattern-matched, like ATL). This reduces boilerplate but limits control over execution order.

Self vs. implicit context. QVT-O uses `self` to refer to the source element within a mapping. JjTL uses an implicit context: properties of the source element are accessed without qualification. This is a significant verbosity reduction but requires clear scoping rules.

7.5 Feature Comparison Matrix

Feature	ATL	ETL	QVT-R	QVT-O	JjTL
Declarative rules	✓	–	✓	–	✓
Imperative blocks	✓	✓	–	✓	Limited[a]
Bidirectional	–	–	✓	–	✓[e]
Automatic trace	✓	✓	✓	✓	✓
Invertibility tracking	–	–	–	–	✓
Guard conditions	✓	✓	✓[b]	✓	✓
Multiplicity	–	–	–	–	✓
Null-safe navigation	–	✓[c]	–	–	✓
Naming conventions	–	–	–	–	✓
Interactive (user input)	–	–	–	–	✓
Set comprehension (`forall`)	–	–	–	–	✓
Rule inheritance	–	✓	–	✓[d]	–
Expression language	OCL	EOL	OCL	OCL	JjEL

[a] Via `forall...do` in expressions. [b] Via `when`/`where` clauses. [c] Since Epsilon 2.1. [d] Via mapping inheritance. [e] Declarative core is bidirectional; also has a unidirectional extension.

7.6 Expression Language Comparison

A critical differentiator is the expression sub-language. We compare equivalent expressions across OCL (used by ATL, QVT-R, QVT-O), EOL (used by ETL), and JjEL (used by JjTL):

Task	OCL	EOL	JjEL
Public attr. names	`self.attr->select(a|a.isPublic)->collect(a|a.name)`	`self.attr.select(a|a.isPublic).collect(a|a.name)`	`attr such that a.isPublic: a.name`
Exists string attr	`self.attr->exists(a|a.type.name='String')`	`self.attr.exists(a|a.type.name='String')`	`attr such that a.type.name='String'`
Abstract ⇒ has children	`self.isAbstract implies self.subClasses->notEmpty()`	`self.isAbstract implies self.subClasses.notEmpty()`	`isAbstract implies subClasses.isNotEmpty`
Null-safe name	`if self.parent.oclIsUndefined() then 'none' else self.parent.name endif`	`self.parent?.name ?: 'none'`	`parent?.name ?? "none"`
To snake_case	(no native support)	(no native support)	`name.snakeCase()`

Key observations:

1. JjEL's `forall` replaces the `->select()->collect()` chain with a single construct.
2. OCL's null handling requires verbose `oclIsUndefined()` checks.
3. Naming convention methods (`snakeCase`, `camelCase`) are unique to JjEL.
4. JjEL uses implicit context (no `self`), reducing token count.

8 Transformation Examples

We present three transformations in parallel across ATL, ETL, and JjTL, each demonstrating different language features.

8.1 Example 1: UML Class to Relational Table

This classic transformation maps UML classes to relational tables: each non-abstract class becomes a table, each single-valued attribute becomes a column, and each multi-valued attribute becomes a junction table.

8.1.1 ATL

Listing 17: Class2Relational in ATL

```
module Class2Relational;
create OUT : Relational from IN : Class;

helper context Class!Attribute def: isSingleValued() : Boolean =
    not self.multiValued;

rule Class2Table {
    from
        c : Class!Class (
            not c.isAbstract
        )
    to
        t : Relational!Table (
            name <- c.name,
            col  <- c.attr->select(a | a.isSingleValued())
                          ->collect(a | thisModule.Attr2Col(a))
                          ->union(Sequence{key}),
            key  <- Set{key}
        ),
        key : Relational!Column (
            name <- 'Id'
        )
}

lazy rule Attr2Col {
    from
        a : Class!Attribute
    to
        c : Relational!Column (
            name <- a.name,
            type <- a.type.name
        )
}

rule MultiAttr2Table {
    from
        a : Class!Attribute (
            a.multiValued
        )
    to
        t : Relational!Table (
            name <- a.owner.name + '_' + a.name,
            col  <- Sequence{id, val}
        ),
        id : Relational!Column (
            name <- 'Id'
        ),
        val : Relational!Column (
            name <- a.name,
```

```
            type <- a.type.name
        )
}
```

8.1.2 ETL

Listing 18: Class2Relational in ETL

```
rule Class2Table
    transform c : Class!Class
    to t : Relational!Table {

    guard : not c.isAbstract

    t.name = c.name;

    // Primary key
    var pk = new Relational!Column;
    pk.name = "Id";
    pk.type = "INTEGER";
    t.col.add(pk);

    // Single-valued attributes
    for (a in c.attr.select(a | not a.multiValued)) {
        var col = new Relational!Column;
        col.name = a.name;
        col.type = a.type.name;
        t.col.add(col);
    }
}

rule MultiAttr2Table
    transform a : Class!Attribute
    to t : Relational!Table {

    guard : a.multiValued

    t.name = a.owner.name + "_" + a.name;

    var id = new Relational!Column;
    id.name = "Id";
    t.col.add(id);

    var val = new Relational!Column;
    val.name = a.name;
    val.type = a.type.name;
    t.col.add(val);
}
```

8.1.3 JjTL

Listing 19: Class2Relational in JjTL

```
transformation Class2Relational

from ClassMM
to   RelationalMM
```

```

# Non-abstract classes become tables
Class -> Table where { not isAbstract } {
    name := name.snakeCase()

    # Primary key column
    -> col {
        -> Column {
            name := "id"
            type := "INTEGER"
        }
    }

    # Single-valued attributes become columns
    -> col {
        forall a in attributes such that not a.multiValued
        -> Column {
            name := a.name.snakeCase()
            type := a.type.name
        }
    }
}

# Multi-valued attributes become junction tables
Attribute -> Table where { multiValued } {
    name := owner.name.snakeCase() + "_" + name.snakeCase()

    -> col {
        -> Column {
            name := "id"
        }
    }
    -> col {
        -> Column {
            name := name.snakeCase()
            type := type.name
        }
    }
}
```

8.2 Example 2: State Machine to Petri Net

This transformation converts a state machine (states and transitions) into a Petri net (places, transitions, and arcs).

8.2.1 ATL

Listing 20: StateMachine2PetriNet in ATL

```
module SM2PN;
create OUT : PetriNet from IN : StateMachine;

rule State2Place {
    from
        s : StateMachine!State
    to
        p : PetriNet!Place (
```

```
            name   <- s.name,
            tokens <- if s.isInitial then 1 else 0 endif
        )
}

rule Transition2Transition {
    from
        t : StateMachine!Transition
    to
        pt : PetriNet!Transition (
            name <- t.label
        ),
        inArc : PetriNet!Arc (
            source    <- t.source,  -- resolved via trace
            target    <- pt,
            weight    <- 1
        ),
        outArc : PetriNet!Arc (
            source    <- pt,
            target    <- t.target,  -- resolved via trace
            weight    <- 1
        )
}
```

8.2.2 ETL

Listing 21: StateMachine2PetriNet in ETL

```
rule State2Place
    transform s : SM!State
    to p : PN!Place {

    p.name   = s.name;
    p.tokens = if (s.isInitial) 1 else 0;
}

rule Transition2Transition
    transform t : SM!Transition
    to pt : PN!Transition {

    pt.name = t.label;

    // Input arc: from source place to this transition
    var inArc = new PN!Arc;
    inArc.source = t.source.equivalent();
    inArc.target = pt;
    inArc.weight = 1;

    // Output arc: from this transition to target place
    var outArc = new PN!Arc;
    outArc.source = pt;
    outArc.target = t.target.equivalent();
    outArc.weight = 1;
}
```

8.2.3 JjTL

Listing 22: StateMachine2PetriNet in JjTL

```
transformation StateMachine2PetriNet

from StateMachineMM
to   PetriNetMM

State -> Place {
    tokens := if isInitial then 1 else 0
}

Transition -> Transition [*] {
    name := label

    # Input arc: source state -> this transition
    -> inputArcs {
        -> Arc {
            source := source
            weight := 1
        }
    }

    # Output arc: this transition -> target state
    -> outputArcs {
        -> Arc {
            target := target
            weight := 1
        }
    }
}
```

Comparison notes.

- ATL and JjTL both use implicit trace resolution: when a source element (e.g., a `State`) is assigned to a target property that expects the corresponding target type (e.g., a `Place`), the engine automatically resolves the trace link. This keeps the mapping syntax clean and declarative.
- ETL requires explicit `.equivalent()` calls to resolve trace links.
- The `if...then...else` expression in JjTL reads naturally and uses JjEL conditional semantics, while ATL requires the verbose `if...then...else...endif` with OCL syntax.

8.3 Example 3: UML to Java (Structural)

This example maps UML classes to Java class definitions, demonstrating naming conventions, inheritance, and collection processing.

8.3.1 ATL

Listing 23: UML2Java structural mapping in ATL

```
module UML2Java;
create OUT : Java from IN : UML;

helper context UML!Class def: javaName() : String =
    self.name;
    -- Note: no native camelCase/PascalCase in OCL
```

```

helper context UML!Attribute def: getterName() : String =
    'get' + self.name;  -- cannot capitalize first letter natively

rule Class2JavaClass {
    from
        c : UML!Class
    to
        j : Java!JavaClass (
            name       <- c.javaName(),
            isAbstract <- c.isAbstract,
            superClass <- c.superClass,  -- trace-resolved
            fields     <- c.attributes->select(a |
                               not a.isDerived),
            methods    <- c.attributes->select(a |
                               not a.isDerived)
                               ->collect(a | thisModule.Attr2Getter(a))
        )
}

lazy rule Attr2Getter {
    from
        a : UML!Attribute
    to
        m : Java!Method (
            name       <- a.getterName(),
            returnType <- a.type,
            body       <- 'return this.' + a.name + ';'
        )
}
```

8.3.2 ETL

Listing 24: UML2Java structural mapping in ETL

```
rule Class2JavaClass
    transform c : UML!Class
    to j : Java!JavaClass {

    j.name       = c.name;
    j.isAbstract = c.isAbstract;
    j.superClass ::= c.superClass;

    for (a in c.attributes.select(a | not a.isDerived)) {
        var field = new Java!Field;
        field.name = a.name;
        field.type ::= a.type;
        j.fields.add(field);

        var getter = new Java!Method;
        getter.name = "get" + a.name.firstToUpperCase();
        getter.returnType ::= a.type;
        getter.body = "return this." + a.name + ";";
        j.methods.add(getter);
    }
}
```

8.3.3 JjTL

Listing 25: UML2Java structural mapping in JjTL

```
transformation UML2Java

from UML_MM
to   JavaMM

Class -> JavaClass {
    name := name.pascalCase()
    isAbstract := isAbstract
    superClass := superClass

    # Fields from non-derived attributes
    -> fields {
        forall a in attributes such that not a.isDerived
        -> Field {
            name := a.name.camelCase()
            type := a.type
        }
    }

    # Getter methods
    -> methods {
        forall a in attributes such that not a.isDerived
        -> Method {
            name := "get" + a.name.pascalCase()
            returnType := a.type
            body := "return this." + a.name.camelCase() + ";"
        }
    }
}
```

Comparison notes.

- JjTL's native `pascalCase()` and `camelCase()` methods eliminate the need for manual string manipulation helpers that ATL and ETL require.
- ATL needs a lazy rule (`Attr2Getter`) invoked via `thisModule.Attr2Getter(a)`; JjTL uses inline object creation.
- ETL's imperative loop with explicit `new` and `.add()` is more verbose than JjTL's declarative `forall` with nested object creation.
- OCL's string handling is the weakest: there is no native method to capitalize the first letter, convert to camelCase, etc.

8.4 Example 4: Validation Invariants

This example demonstrates JjEL's expressiveness for model validation, compared to OCL.

Invariant	OCL	JjEL
All classes have ≥1 attribute	`Class.allInstances()->forAll(c | c.attributes->notEmpty())`	`forall c in classes: c.attributes.isNotEmpty`
Abstract ⇒ has subclasses	`self.isAbstract implies self.subClasses->notEmpty()`	`isAbstract implies subClasses.isNotEmpty`
Unique class names	`Class.allInstances()->isUnique(c | c.name)`	`(forall c in classes: c.name).distinct().size == classes.size`
No attribute with null type	`self.attributes->forAll(a | not a.type.oclIsUndefined())`	`forall a in attributes: a.type != null`
Public attributes ⇒ typed	`self.attributes->select(a | a.isPublic)->forAll(a | not a.type.oclIsUndefined())`	`forall a in attributes such that a.isPublic: a.type != null`

In JjEL, the same `forall` construct serves for both selection and projection, while OCL requires different operations (`->forAll()` for assertion, `->select()` for filtering, `->collect()` for mapping). Note also that in JjEL validation, `forall` with `:` returns a set of boolean values; the invariant holds if all values are `true`. An explicit `.all(x => x)` or wrapping with `forall ... such that` can make the boolean intent more explicit.

8.5 Example 5: Guard Conditions

Guards in transformation rules demonstrate how expression languages handle complex selection criteria.

Guard	ATL (OCL)	ETL (EOL)	JjTL (JjEL)
Non-abstract with attributes	`not c.isAbstract and c.attr->notEmpty()`	`not c.isAbstract and c.attr.notEmpty()`	`not isAbstract and attributes.isNotEmpty`
Has a string attribute	`c.attr->exists(a | a.type.name = 'String')`	`c.attr.exists(a | a.type.name = 'String')`	`exists a in attributes | a.type == "String"`
All references are navigable	`c.refs->forAll(r | r.isNavigable)`	`c.refs.forAll(r | r.isNavigable)`	`forall r in references: r.isNavigable`

9 Discussion

9.1 Strengths of the JjTL/JjEL Approach

1. **Conceptual unification via `forall`**: By giving `forall` set-theoretic semantics, JjEL replaces three distinct concepts (filter, map, logical quantification) with one. This is a genuine reduction in cognitive load, not merely syntactic sugar.
2. **Domain-native operations**: Built-in naming convention methods (`camelCase`, `snakeCase`, etc.) and null-safe navigation address the most common pain points in model transformations, which no other transformation language handles natively.
3. **Invertibility tracking**: JjTL's automatic classification of attribute mappings as invertible or non-invertible is unique among the compared languages and enables downstream applications like bidirectional synchronization and transformation debugging.
4. **Interactive transformations**: The `prompt`/`input` mechanism supports semi-automated scenarios that other languages cannot express.

5. **Single execution model**: Unlike ATL's two-phase matched rules or QVT-R's complex enforcement semantics, JjTL's execution model is straightforward: match, create, bind.

9.2 Limitations and Trade-offs

1. **Limited imperative support**: JjTL's `do` blocks provide basic imperative capability, but lack the full expressiveness of ATL's called rules or QVT-O's mapping operations for complex procedural logic.
2. **No rule inheritance**: Unlike ETL's `extends` and QVT-O's mapping inheritance, JjTL rules are flat. Complex transformations with shared logic across rules must use helper functions.
3. **No explicit entry point**: The automatic "fire all matching rules" execution model (like ATL) limits control over rule ordering. QVT-O's explicit `main()` provides more control.
4. **Maturity**: ATL and ETL have decades of academic use, extensive example repositories, and battle-tested implementations. JjTL/JjEL are new and will require the same maturation process.

9.3 Open Design Points

Several aspects of JjEL are under active review:

1. `if/then/else` **syntax**: The conditional has a procedural flavor. Alternatives like `when/then/otherwise` have been considered for their more declarative reading.
2. **String interpolation**: The lexer supports `${...}` syntax but the parser does not yet produce interpolated string nodes. This is planned as a future enhancement.
3. **Closure method**: A generic `closure()` method for transitive navigation would replace hardcoded properties like `allSuperclasses` and `allSubclasses`.
4. `forall` **nesting**: The readability and semantics of nested `forall` expressions need further validation.

9.4 JjScript in the MDE Scripting Landscape

JjScript occupies a design space adjacent to, but distinct from, the transformation languages compared in section 7. Its closest analogue in the literature is **EOL** (Epsilon Object Language), the imperative base language of the Epsilon platform. Both are command-oriented, imperative, and operate on model elements. However, three differences are significant:

1. **Abstraction level.** EOL operates on model instances (M1): it navigates, queries, and modifies elements conforming to a metamodel. JjScript operates on the *metamodel itself* (M2): it creates and modifies classes, attributes, references, and packages. This is a fundamental difference in scope—JjScript is a metamodel engineering tool, not a model manipulation language.
2. **Expression delegation.** EOL has its own expression evaluator (a Java-like syntax with closures). JjScript delegates all expressions to JjEL, ensuring a single expression syntax across the entire Jjodel language family. This reduces cognitive load for users who work with both JjTL and JjScript.
3. **AI integration.** JjScript is designed as a generation target for AI-mediated interaction: the Jjodie assistant translates natural-language requests into JjScript commands. No published MDE scripting language has been designed with this integration pattern.

QVT-O shares JjScript's imperative character but targets M2M transformation (source model → target model), not metamodel editing. QVT-O's `mapping` operations, `init`/`end` sections, and OCL-based expressions are considerably more complex than JjScript's verb-first command syntax. JjScript trades QVT-O's generality for accessibility: a user can productively use JjScript without reading a specification—the command names are self-documenting, and the autocompletion engine guides syntax.

10 Conclusion

This document has presented the Jjodel language family—JjEL, JjTL, JjModal, JjLet, and JjScript—as a set of complementary domain-specific languages for Model-Driven Engineering. The key contribution is JjEL's set-theoretic approach to expressions: by reinterpreting `forall` as a set comprehension rather than a logical assertion, we achieve a significant reduction in cognitive load while maintaining the declarative character valued in the MDE community. JjScript complements the declarative languages by providing an imperative, command-based interface for direct metamodel manipulation, completing the purity–effect spectrum from pure expressions (JjEL) through declarative transformations (JjTL) to imperative scripting (JjScript).

The comparative analysis with ATL, EOL/ETL, QVT-R, and QVT-O reveals that each language occupies a distinct design point:

- ATL combines declarative rules with OCL expressions but suffers from two-phase execution complexity and limited tooling.
- ETL prioritizes developer familiarity via an imperative style but sacrifices the declarative invertibility that enables advanced transformation analysis.
- QVT-R achieves full declarativity and bidirectionality at the cost of practical usability.
- QVT-O provides a pragmatic imperative approach with OCL's verbosity.

JjTL/JjEL targets the space between ATL's declarative elegance and ETL's practical simplicity, aiming for a language that is *declarative enough* for transformation analysis (invertibility, traceability) while being *intuitive enough* for adoption by domain experts who may not have a formal logic background.

A JjEL Grammar (Complete)

Listing 26: JjEL formal grammar in EBNF

```
expression     = forallExpr | existsExpr | withExpr | ifThenElse

forallExpr     = 'forall' IDENT 'in' expression
                   ('such' 'that' expression)?
                   (':' expression)?
                   ('do' expression)?

existsExpr     = 'exists' IDENT 'in' expression
                   (':' | 'such' 'that') expression

withExpr       = 'with' expression 'do' expression

ifThenElse     = 'if' expression 'then' expression ('else' expression)?
               | nullCoalesce

nullCoalesce   = implies ('??' implies)*
```

```
17
18 implies        = logicalOr ('implies' logicalOr)?
19
20 logicalOr      = logicalAnd ('or' logicalAnd)*
21
22 logicalAnd     = equality ('and' equality)*
23
24 equality       = comparison (('==' | '!=') comparison)*
25
26 comparison     = isType (('<' | '>' | '<=' | '>=') isType)*
27
28 isType         = addition ('is' IDENT)?
29
30 addition       = multiplication (('+' | '-') multiplication)*
31
32 multiplication = unary (('*' | '/' | '%') unary)*
33
34 unary          = 'not' unary
35                | '-' unary
36                | postfix
37
38 postfix        = primary (
39                    '.' IDENT ('(' argList ')')?
40                  | '?.' IDENT ('(' argList ')')?
41                  | '[' expression ']'
42                  )*
43
44 primary        = literal | IDENT | lambda | arrayLiteral
45                | '(' expression ')'
46
47 lambda         = IDENT '=>' expression
48                | '(' IDENT (',' IDENT)* ')' '=>' expression
49
50 arrayLiteral   = '[' (expression (',' expression)*)? ']'
51
52 argList        = (expression (',' expression)*)?
53
54 literal        = STRING | NUMBER | 'true' | 'false' | 'null'
```

B JjEL Built-in Methods (Complete)

B.1 String Methods (36)

Method	Parameters	Description
`toUpper()`	–	Uppercase
`toLower()`	–	Lowercase
`capitalize()`	–	First letter uppercase
`uncapitalize()`	–	First letter lowercase
`camelCase()`	–	`"my name"` → `"myName"`
`pascalCase()`	–	`"my name"` → `"MyName"`
`snakeCase()`	–	`"myName"` → `"my_name"`
`kebabCase()`	–	`"myName"` → `"my-name"`
`trim()`	–	Remove surrounding whitespace
`trimStart()`	–	Remove leading whitespace

Method	Parameters	Description
`trimEnd()`	–	Remove trailing whitespace
`padStart(n, ch)`	length, char	Pad start to length
`padEnd(n, ch)`	length, char	Pad end to length
`contains(s)`	substring	Contains check
`startsWith(s)`	prefix	Prefix check
`endsWith(s)`	suffix	Suffix check
`indexOf(s)`	substring	First occurrence index
`lastIndexOf(s)`	substring	Last occurrence index
`matches(r)`	regex	Regex match
`replace(o, n)`	old, new	Replace first
`replaceAll(o, n)`	old, new	Replace all
`substring(s, e?)`	start, end?	Extract substring
`slice(s, e?)`	start, end?	Slice (negative indices)
`split(sep)`	separator	Split to array
`repeat(n)`	count	Repeat string
`reverse()`	–	Reverse string
`length`	–	Character count
`isEmpty`	–	Length == 0
`isNotEmpty`	–	Length > 0
`isBlank`	–	Empty or whitespace only
`isNotBlank`	–	Not blank
`charAt(i)`	index	Character at position
`toNumber()`	–	Parse as number
`toInt()`	–	Parse as integer
`quote()`	–	Wrap in double quotes
`format(args...)`	values	String formatting with placeholders

B.2 Collection Methods (31)

While `filter()` and `map()` remain available, the idiomatic approach uses `forall`. The full collection method list is:

Method	Parameters	Description
`filter(fn)`	predicate	Filter elements (deprecated; use `forall`)
`map(fn)`	lambda	Transform elements (deprecated; use `forall`)
`sum()`	–	Numeric sum
`avg()`	–	Numeric average
`min()`	–	Minimum value
`max()`	–	Maximum value
`count()`	–	Element count
`first`	–	First element
`last`	–	Last element
`at(i)`	index	Element at index
`indexOf(x)`	item	Index of element
`size`	–	Collection size
`length`	–	Alias for size
`isEmpty`	–	Size == 0
`isNotEmpty`	–	Size > 0
`contains(x)`	item	Membership test

Method	Parameters	Description
`distinct()`	–	Remove duplicates
`distinctBy(fn)`	lambda	Remove duplicates by key
`sortBy(fn)`	lambda	Sort ascending
`sortByDescending(fn)`	lambda	Sort descending
`reverse()`	–	Reverse order
`flatten()`	–	Flatten nested arrays
`flatMap(fn)`	lambda	Map + flatten
`groupBy(fn)`	lambda	Group by key
`take(n)`	count	First n elements
`skip(n)`	count	Skip first n
`takeWhile(fn)`	predicate	Take while true
`skipWhile(fn)`	predicate	Skip while true
`join(sep)`	separator	Join to string
`all(fn)`	predicate	All satisfy?
`any(fn)`	predicate	Any satisfies?
`none(fn)`	predicate	None satisfy?

C JjTL Token Types

The JjTL lexer recognises 44 token types. Since JjTL embeds JjEL expressions, its token set is a superset of the JjEL tokens.

Category	Token	Lexeme
JjTL keywords	TRANSFORMATION	transformation
	FROM	from
	TO	to
	WHERE	where
	HELPER	helper
	LET	let
Interactive	ALERT	alert
	NOTIFY	notify
	PROMPT	prompt
	INPUT	input
	CONFIRM	confirm
Iteration	FORALL	forall
	EXISTS	exists
	IN	in
	SUCH	such
	THAT	that
	DO	do
JjEL keywords	IF / THEN / ELSE	if, then, else
	AND / OR / NOT	and, or, not
	IS	is
	IMPLIES	implies
	WITH	with
	TRUE / FALSE	true, false
	NULL	null
Literals	STRING	"hello", 'world'
	NUMBER	42, 3.14
	BOOLEAN	true, false
	IDENTIFIER	name, State
	DOLLAR_IDENT	$prefix, $ok
Operators	ARROW	->
	FAT_ARROW	=>
	ASSIGN	:=
	DOT	.
	QUESTION_DOT	?.
	NULL_COALESCE	??
	EQUALS	=
	COLON	:
Comparison	EQUALS_EQUALS / NOT_EQUALS	==, !=
	LESS_THAN / GREATER_THAN	<, >
	LESS_EQUAL / GREATER_EQUAL	<=, >=
	PLUS / MINUS	+, -
	STAR / SLASH / PERCENT	*, /, %
	PIPE	|
Delimiters	LBRACE / RBRACE	{, }
	LPAREN / RPAREN	(,)
	LBRACKET / RBRACKET	[,]
Punctuation	COMMA	,
	HASH	#
Special	COMMENT	- ... or # ...
	NEWLINE	end of line
	WHITESPACE	spaces, tabs
	EOF	end of input